AI & Machine Learning Mastery

Complete Program Syllabus

Table of Contents

Program Overview

Modules at a Glance

Module 1: Mathematics & Python for AI

1.1 Python for Data Science
1.2 Linear Algebra
1.3 Calculus for ML
1.4 Probability & Statistics
1.5 Optimization Basics

Module 2: Classical Machine Learning

2.1 ML Fundamentals
2.2 Regression
2.3 Classification
2.4 Ensemble Methods
2.5 Unsupervised Learning
2.6 Feature Engineering & Model Selection

Module 3: Deep Learning & Neural Nets

3.1 Neural Network Foundations
3.2 Training Deep Networks

Shrinovatech

3.3 Frameworks
3.4 Convolutional Neural Networks (CNNs) — Intro
3.5 Sequence Models — Intro
3.6 Training at Scale

Module 4: Computer Vision & NLP

4.1 Computer Vision
4.2 Natural Language Processing
4.3 Multimodal Foundations

Module 5: Generative AI & LLMs

5.1 Generative Model Foundations
5.2 Large Language Models
5.3 Prompt Engineering
5.4 Retrieval-Augmented Generation (RAG)
5.5 Fine-Tuning & Adaptation
5.6 Agents & Tool Use
5.7 Working with APIs

Module 6: Production AI Deployment

6.1 MLOps Fundamentals
6.2 Model Serving
6.3 Scaling & Infrastructure
6.4 Optimization for Production
6.5 Monitoring & Reliability
6.6 CI/CD for ML
6.7 Security & Governance

Shrinovatech

Program Overview

This program takes learners from core mathematical and programming foundations through classical machine learning, deep learning, computer vision and NLP, generative AI and large language models, and finally into production deployment of AI systems. Each module builds directly on the last and ends with a hands-on project.

Modules at a Glance

No	Module	Focus	Duration
1	Mathematics & Python for AI	Math & coding foundations	2–3 weeks
2	Classical Machine Learning	Core supervised/unsupervised algorithms	3–4 weeks
3	Deep Learning & Neural Nets	Neural network theory & frameworks	3–4 weeks
4	Computer Vision & NLP	Vision and language deep learning	4 weeks
5	Generative AI & LLMs	Generative models, LLMs, RAG, agents	4 weeks
6	Production AI Deployment	MLOps, serving, monitoring at scale	3 weeks

Srinivasan

Module 1: Mathematics & Python for AI

Goal: *Build the mathematical intuition and coding fluency needed for every later module.*

1.1 Python for Data Science

- Python fundamentals refresher (data structures, functions, OOP basics)
- NumPy: arrays, broadcasting, vectorized operations
- Pandas: dataframes, cleaning, merging, groupby
- Matplotlib/Seaborn: exploratory data visualization
- Jupyter/Colab workflow, virtual environments

1.2 Linear Algebra

- Vectors, matrices, tensors
- Matrix multiplication, transpose, inverse
- Eigenvalues & eigenvectors
- Singular Value Decomposition (SVD)
- Applications: PCA, image representation

1.3 Calculus for ML

- Derivatives & partial derivatives
- Gradients and the chain rule
- Gradient descent (batch, stochastic, mini-batch)
- Convexity and optimization landscapes

1.4 Probability & Statistics

- Probability distributions (Bernoulli, Gaussian, Binomial, Poisson)
- Bayes' theorem and conditional probability
- Descriptive statistics, hypothesis testing
- Maximum Likelihood Estimation (MLE)
- Correlation vs. causation

1.5 Optimization Basics

- Loss functions and cost surfaces
- Learning rate, convergence

Shrinovatech

- Regularization concepts (L1/L2)

Module Project: Implement gradient descent from scratch to fit a linear regression model without using ML libraries.

Module 2: Classical Machine Learning

Goal: *Master the core supervised/unsupervised algorithms and the ML workflow.*

2.1 ML Fundamentals

- Supervised vs. unsupervised vs. reinforcement learning
- Train/validation/test splits, cross-validation
- Bias-variance tradeoff, overfitting/underfitting

2.2 Regression

- Linear & polynomial regression
- Ridge, Lasso, Elastic Net
- Evaluation metrics: MSE, RMSE, R^2

2.3 Classification

- Logistic regression
- k-Nearest Neighbors (KNN)
- Decision Trees & Random Forests
- Support Vector Machines (SVM)
- Naive Bayes
- Evaluation: accuracy, precision, recall, F1, ROC-AUC, confusion matrix

2.4 Ensemble Methods

- Bagging vs. boosting
- Gradient Boosting (XGBoost, LightGBM, CatBoost)
- Stacking

2.5 Unsupervised Learning

- K-Means, Hierarchical clustering, DBSCAN
- Dimensionality reduction: PCA, t-SNE, UMAP

Shrinovatech

- Anomaly detection

2.6 Feature Engineering & Model Selection

- Encoding categorical variables, scaling/normalization
- Feature selection techniques
- Hyperparameter tuning: Grid Search, Random Search, Bayesian Optimization
- Handling imbalanced datasets

Module Project: End-to-end ML pipeline on a tabular dataset — from EDA to a tuned, evaluated model with a written analysis.

Module 3: Deep Learning & Neural Nets

Goal: *Understand how neural networks learn and how to build/train them with modern frameworks.*

3.1 Neural Network Foundations

- Perceptrons, multi-layer perceptrons (MLPs)
- Activation functions (ReLU, sigmoid, tanh, softmax, GELU)
- Forward propagation & backpropagation
- Weight initialization strategies

3.2 Training Deep Networks

- Loss functions (cross-entropy, MSE, etc.)
- Optimizers: SGD, Momentum, RMSProp, Adam
- Batch normalization, dropout, regularization
- Learning rate schedules, early stopping
- Vanishing/exploding gradients

3.3 Frameworks

- PyTorch fundamentals: tensors, autograd, nn.Module
- TensorFlow/Keras overview
- Building custom training loops

Shrinovatech

3.4 Convolutional Neural Networks (CNNs) — Intro

- Convolution, pooling, feature maps
- Classic architectures: LeNet, AlexNet, VGG, ResNet (conceptual intro)

3.5 Sequence Models — Intro

- RNNs, LSTMs, GRUs (conceptual intro; deep dive in Module 4)

3.6 Training at Scale

- GPUs/TPUs, mixed precision training
- Data augmentation
- Debugging and monitoring training runs (TensorBoard/W&B)

Module Project: Build and train a neural network from scratch (NumPy) for digit classification, then reproduce it in PyTorch/TensorFlow.

Module 4: Computer Vision & NLP

Goal: *Apply deep learning to images and language, using both classical and modern architectures.*

4.1 Computer Vision

- CNN architectures in depth: ResNet, Inception, EfficientNet
- Transfer learning & fine-tuning pretrained models
- Object detection: YOLO, Faster R-CNN
- Image segmentation: U-Net, Mask R-CNN
- Vision Transformers (ViT)
- Image augmentation pipelines

4.2 Natural Language Processing

- Text preprocessing: tokenization, stemming/lemmatization
- Word embeddings: Word2Vec, GloVe
- Sequence models for text: RNN/LSTM/GRU for classification & tagging
- Attention mechanism — intuition and math

Shrinovatech

- The Transformer architecture (encoder-decoder, self-attention, positional encoding)
- Named Entity Recognition (NER), sentiment analysis, text classification

4.3 Multimodal Foundations

- Image captioning
- CLIP-style vision-language models (conceptual intro)

Module Project: Build an image classifier via transfer learning AND a text classifier using a Transformer-based model (e.g., fine-tuning BERT).

Module 5: Generative AI & LLMs

Goal: *Understand how large language and generative models work, and how to build applications on top of them.*

5.1 Generative Model Foundations

- Autoencoders & Variational Autoencoders (VAEs)
- Generative Adversarial Networks (GANs)
- Diffusion models (conceptual + practical intro)

5.2 Large Language Models

- Transformer architecture deep dive (self-attention, multi-head attention, positional encodings)
- Pretraining vs. fine-tuning
- Tokenization (BPE, SentencePiece)
- Scaling laws, emergent capabilities
- Instruction tuning & RLHF (conceptual)

5.3 Prompt Engineering

- Zero-shot, few-shot, chain-of-thought prompting
- System prompts and prompt templates
- Structured output generation

5.4 Retrieval-Augmented Generation (RAG)

- Embeddings and vector databases (FAISS, Pinecone, Chroma)

Shrinovatech

- Chunking strategies
- Building a RAG pipeline end-to-end

5.5 Fine-Tuning & Adaptation

- Full fine-tuning vs. parameter-efficient methods (LoRA, QLoRA, adapters)
- Dataset preparation for fine-tuning

5.6 Agents & Tool Use

- Function calling / tool use
- Agentic workflows and orchestration
- Multi-agent systems (conceptual intro)

5.7 Working with APIs

- Using LLM provider APIs (e.g., Anthropic, OpenAI)
- Cost/latency tradeoffs, streaming responses

Module Project: Build a RAG-based Q&A application over a custom document set, with an agent that can call at least one external tool.

Module 6: Production AI Deployment

Goal: *Take models from notebooks to reliable, monitored, scalable production systems.*

6.1 MLOps Fundamentals

- ML lifecycle: data → training → deployment → monitoring → retraining
- Experiment tracking (MLflow, Weights & Biases)
- Model versioning and reproducibility

6.2 Model Serving

- REST/gRPC APIs for model inference (FastAPI, Flask)
- Batch vs. real-time inference
- Model serialization (ONNX, TorchScript, SavedModel)
- Serving frameworks: TorchServe, TensorFlow Serving, Triton

Shrino Tech

6.3 Scaling & Infrastructure

- Containerization with Docker
- Orchestration with Kubernetes basics
- Cloud ML platforms (AWS SageMaker, GCP Vertex AI, Azure ML)
- Serverless inference, autoscaling

6.4 Optimization for Production

- Model quantization, pruning, distillation
- Latency and throughput optimization
- Caching strategies for LLM apps

6.5 Monitoring & Reliability

- Data drift and model drift detection
- Logging, alerting, A/B testing
- Guardrails and safety checks for generative AI systems

6.6 CI/CD for ML

- Automated testing for ML pipelines
- CI/CD pipelines for model deployment
- Rollback strategies

6.7 Security & Governance

- Data privacy considerations
- Responsible AI practices, bias auditing
- Compliance basics (GDPR-type considerations)

Capstone Project: Deploy an end-to-end AI application (choose CV, NLP, or LLM-based) to a live, monitored production environment — including API, containerization, and a monitoring dashboard

Aminovatech